complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + reduce() combine values

filter() null checks + reduce() combine values

filter() comparisons + reduce() combine values

filter() method calls returning boolean + reduce() combine values

distinct() rely on equals() / hashcode() + reduce() combine values

distinct() immutable objects + reduce() combine values

map() transform objects + reduce() combine values

map() create new objects + reduce() combine values

map() call pure functions + reduce() combine values

flatmap() return stream + reduce() combine values

flatmap() split data + reduce() combine values

flatmap() expand collections + reduce() combine values

sorted() natural ordering + reduce() combine values

sorted() custom comparator + reduce() combine values

sorted() multi-field compare + reduce() combine values

peek() logging + reduce() combine values

peek() debugging + reduce() combine values

peek() metrics + reduce() combine values

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **every requested Stream combination**, where:

- **One primary intermediate operation** is demonstrated
- **reduce() is the terminal operation**
- **Goal = combine values** (sum, concat, merge)
- Java **8+ compatible**
- Clean, deterministic, interview-ready
- No collectors, no forEach

 **Mental model**
reduce(identity, accumulator) ⇒ **Combine stream elements into ONE result**

---

**1 filter() boolean checks + reduce() Combine values**

```
import java.util.*;

public class FilterBooleanReduce {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      int sum =
            numbers.stream()
                  .filter(n -> n % 2 == 0)
                  .reduce(0, Integer::sum);

      System.out.println(sum);
   }
}
```

---

**2 filter() null checks + reduce() Combine values**

```
import java.util.*;

public class FilterNullReduce {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      String combined =
            names.stream()
                  .filter(Objects::nonNull)
                  .reduce("", (a, b) -> a + b);

      System.out.println(combined);
   }
}
```

---

**3 filter() comparisons + reduce() Combine values**

```
import java.util.*;

public class FilterComparisonReduce {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      int total =
            salaries.stream()
                  .filter(s -> s >= 30000)
                  .reduce(0, Integer::sum);

      System.out.println(total);
   }
}
```

---

**4 filter() method call boolean + reduce() Combine values**

```
import java.util.*;

public class FilterMethodReduce {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List<User> users = List.of(
            new User("Anurag", true),
            new User("Amit", true),
            new User("Ravi", false)
      );

      int count =
            users.stream()
                  .filter(User::isActive)
                  .map(u -> 1)
                  .reduce(0, Integer::sum);

      System.out.println(count);
   }
}
```

**5 distinct() equals/hashCode + reduce() Combine values**

```
import java.util.*;

public class DistinctEqualsReduce {

   static class Person {
      String name;

      Person(String name) { this.name = name; }

      @Override
      public boolean equals(Object o) {
         return o instanceof Person p && name.equals(p.name);
      }

      @Override
      public int hashCode() {
         return Objects.hash(name);
      }
   }

   public static void main(String[] args) {
      List<Person> people = List.of(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      int totalLength =
            people.stream()
                  .distinct()
                  .map(p -> p.name.length())
                  .reduce(0, Integer::sum);

      System.out.println(totalLength);
   }
}
```

**6 distinct() immutable objects + reduce() Combine values**

```
import java.util.*;

public class DistinctImmutableReduce {
   public static void main(String[] args) {
      List<String> values = List.of("Java", "Java", "Go");

      int lengthSum =
            values.stream()
                  .distinct()
                  .map(String::length)
                  .reduce(0, Integer::sum);

      System.out.println(lengthSum);
   }
}
```

---

**7 map() transform objects + reduce() Combine values**

```
import java.util.*;

public class MapTransformReduce {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit");

      int totalChars =
            names.stream()
                  .map(String::toUpperCase)
                  .map(String::length)
                  .reduce(0, Integer::sum);

      System.out.println(totalChars);
   }
}
```

---

**8 map() create new objects + reduce() Combine values**

```
import java.util.*;

public class MapNewObjectReduce {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Amit");

      int count =
            names.stream()
                  .map(UserDTO::new)
                  .map(dto -> 1)
                  .reduce(0, Integer::sum);

      System.out.println(count);
   }
}
```

---

**9 map() pure functions + reduce() Combine values**

```
import java.util.*;

public class MapPureFunctionReduce {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);

      int sumOfSquares =
            numbers.stream()
                  .map(MapPureFunctionReduce::square)
                  .reduce(0, Integer::sum);

      System.out.println(sumOfSquares);
   }
}
```

---

**10 flatMap() return Stream + reduce() Combine values**

```
import java.util.*;

public class FlatMapStreamReduce {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      int total =
            lists.stream()
                  .flatMap(List::stream)
                  .reduce(0, Integer::sum);

      System.out.println(total);
   }
}
```

---

**11 flatMap() split data + reduce() Combine values**

```
import java.util.*;

public class FlatMapSplitReduce {
   public static void main(String[] args) {
      List<String> sentences = List.of("Java Stream", "Flat Map");

      int wordCount =
            sentences.stream()
                  .flatMap(s -> Arrays.stream(s.split(" ")))
                  .map(w -> 1)
                  .reduce(0, Integer::sum);

      System.out.println(wordCount);
   }
}
```

---

**12 flatMap() expand collections + reduce() Combine values**

```
import java.util.*;

public class FlatMapExpandReduce {
   public static void main(String[] args) {
      Map<String, List<Integer>> data = Map.of(
            "A", List.of(1, 2),
            "B", List.of(3)
      );

      int sum =
            data.values().stream()
                  .flatMap(Collection::stream)
                  .reduce(0, Integer::sum);

      System.out.println(sum);
   }
}
```

---

**13 sorted() natural ordering + reduce() Combine values**

```
import java.util.*;

public class SortedNaturalReduce {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);

      int total =
            numbers.stream()
                  .sorted()
                  .reduce(0, Integer::sum);

      System.out.println(total);
   }
}
```

---

**14 sorted() custom comparator + reduce() Combine values.**

```
import java.util.*;

public class SortedCustomReduce {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      String combined =
            names.stream()
                  .sorted(Comparator.reverseOrder())
                  .reduce("", (a, b) -> a + b);

      System.out.println(combined);
   }
}
```

---

**15 sorted() multi-field compare + reduce() Combine values**

```
import java.util.*;

public class SortedMultiFieldReduce {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
      List<Employee> employees = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      );

      int ageSum =
            employees.stream()
                  .sorted(Comparator
                        .comparing((Employee e) -> e.name)
                        .thenComparingInt(e -> e.age))
                  .map(e -> e.age)
                  .reduce(0, Integer::sum);

      System.out.println(ageSum);
   }
}
```

---

**16 peek() logging + reduce() Combine values**

```
import java.util.*;

public class PeekLoggingReduce {
   public static void main(String[] args) {
      int sum =
            List.of(1, 2, 3).stream()
                  .peek(n -> System.out.println("Peek: " + n))
                  .reduce(0, Integer::sum);

      System.out.println(sum);
   }
}
```

---

**17 peek() debugging + reduce() Combine values**

```
import java.util.*;

public class PeekDebugReduce {
   public static void main(String[] args) {
      int sum =
            List.of(10, 20, 30).stream()
                  .peek(n -> {
                     if (n == 20) System.out.println("Debug hit");
                  })
                  .reduce(0, Integer::sum);

      System.out.println(sum);
   }
}
```

---

**18 peek() metrics + reduce() Combine values**

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsReduce {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      int sum =
            List.of(5, 10, 15).stream()
                  .peek(n -> counter.incrementAndGet())
                  .reduce(0, Integer::sum);

      System.out.println("Sum: " + sum);
      System.out.println("Processed: " + counter.get());
   }
}
```

---